C. CPU Scheduling — Gantt · TAT · WT (OSNo04 สไลด์ท้าย + วิดีโอ 6.x)

> 📖 **รู้จักส่วนนี้ก่อนอ่าน**
>
> **ความหมาย:** CPU Scheduling = การตัดสินใจว่า process ไหนจะได้ใช้ CPU "ตัวต่อไป" เมื่อหลายงานแย่งกัน — มีหลายอัลกอริทึม เช่น FCFS (มาก่อนได้ก่อน), SJF (งานสั้นก่อน), RR (วนแบ่งเวลา)
>
> **ความสำคัญกับข้อสอบ:** โจทย์วาด Gantt (แผนภูมิช่วงเวลา) หา TAT/WT เป็นข้อคำนวณหลักประจำทุกปี — ฝึกให้ทำข้อละไม่เกิน 2 นาที ใช้เทคนิค "หา TAT ก่อนแล้วลบ BT เสมอ"
>
> **ใช้จริงในคอมพิวเตอร์:** Windows/Linux/macOS ใช้แนวคิด priority + time slice (โควตาเวลาแบบ RR) จริง ๆ ในตัวจัดคิวของเคอร์เนล — เปิด Task Manager เห็น process แย่ง CPU ด้วยกลไกพวกนี้ทั้งหมด

C1. พื้นฐาน + สูตรทุกตัวที่ใช้คิดเลข

- **CPU Scheduler (ตัวจัดคิว CPU)** เลือก process จาก ready queue ให้ใช้ CPU · **Non-preemptive** = รันจนจบ/ขอออกเอง · **Preemptive (ขัดจังหวะ/แย่งได้)** = ขัดกลางคันได้ (หมดเวลา/มีตัวสำคัญกว่ามา)
- **CPU burst** = ช่วงที่ process ใช้ CPU ติดกัน · I/O-bound = burst สั้นถี่ / CPU-bound = burst ยาว

`Turnaround Time (เวลารวมในระบบ): TAT = Completion − Arrival`

เวลารวมตั้งแต่งานมาถึง → เสร็จ

`Waiting Time (เวลารอคิว): WT = TAT − Burst Time (เวลาที่ต้องใช้ CPU)`

🟢 เทคนิคทำเร็ว: หา TAT ก่อนแล้วลบ BT เสมอ — โดยเฉพาะ preemptive ที่งานโดนแทรกหลายรอบ

`Response Time (เวลาตอบสนอง) = เวลาที่งานมาถึง → เริ่มได้ response ครั้งแรก`

เกณฑ์วัดอื่น: CPU utilization ↑, throughput ↑, TAT ↓, WT ↓, response ↓

C2. FCFS — First-Come, First-Served (non-preemptive)

โจทย์ (ตรงจากสไลด์ p.34): P1 (Arrival 0, Burst 24) · P2 (2, 3) · P3 (4, 3)

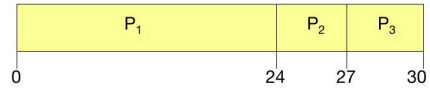


Process	TAT = C − A	WT = TAT − BT
P1	24 − 0 = 24	24 − 24 = **0**
P2	27 − 2 = 25	25 − 3 = **22**
P3	30 − 4 = 26	26 − 3 = **23**
เฉลี่ย	**25**	**(0+22+23)/3 = 15**

> 🎯 **Convoy Effect (อาการคอยตามงานใหญ่)**
>
> งานใหญ่ตัวเดียวบล็อกหน้า งานเล็กทั้งบ้านต่อแถวรอ — สไลด์ p.35 เปรียบเทียบ: ถ้างานสั้นมาก่อน (P2 มา 0, P3 มา 2, P1 มา 4) avg WT เหลือ **1** เท่านั้น!

First-Come, First-Served (FCFS) Scheduling

Process	Arrival Time	Burst Time
P_1	0.0	24
P_2	2.0	3
P_3	4.0	3

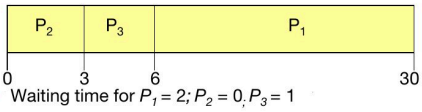


- Waiting time for P_1 = 0; P_2 = 22; P_3 = 23
- Average waiting time: (0 + 22 + 23)/3 = 15

สไลด์ p.34 FCFS

FCFS Scheduling (Cont)

Process	Arrival Time	Burst Time
P_2	0.0	3
P_3	2.0	3
P_1	4.0	24



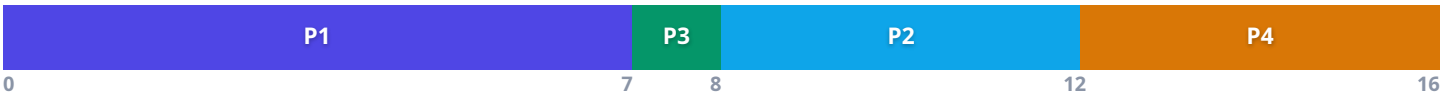
- Waiting time for P_1 = 2; P_2 = 0, P_3 = 1
- Average waiting time: (2 + 0 + 1)/3 = 1
- Much better than previous case
- *Convoy effect* short process behind long process

สไลด์ p.35 FCFS — สลับงานสั้นมาก่อน WT=1

C3. SJF — Shortest Job First (non-preemptive) — OPTIMAL

โจทย์ (p.36): P1 (0, 7) · P2 (2, 4) · P3 (4, 1) · P4 (5, 4) — เมื่อ CPU ว่าง เลือก BT สั้นสุดในคิว ณ ตอนนั้น (เท่ากันเลือกมาก่อน)

1. เวลา 0: มีแค่ P1 → รันจนจบที่ 7 (non-preemptive ใครมาทีหลังรอก่อน)
2. เวลา 7: คิว P2(4), P3(1), P4(4) → สั้นสุด P3 → 7→8
3. เวลา 8: P2(4), P4(4) เท่ากัน → มาก่อนชนะ: P2 → 8→12
4. เวลา 12: P4 → 12→16



Process	Completion	TAT	WT = TAT − BT
P1	7	7	7−7 = **0**
P2	12	10	10−4 = **6**
P3	8	4	4−1 = **3**
P4	16	11	11−4 = **7**
เฉลี่ย WT	**—**		**(0+6+3+7)/4 = 4**

📌 SJF คือ optimal

ให้ avg WT ต่ำสุดจากชุดงานหนึ่ง ๆ แต่ใช้จริงยากเพราะ OS ต้อง**ทำนาย**ความยาว CPU burst ถัดไป

C4. SRTF (งานที่เหลือสั้นสุดทำก่อน) — Shortest Remaining Time First (Preemptive SJF) 🔥

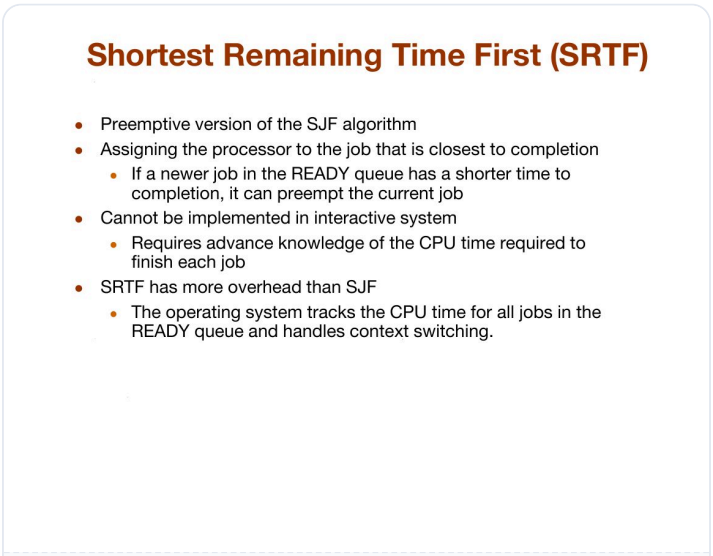
กติกา: **งานใหม่มาถึง → เทียบ BT ของงานใหม่ กับ Remaining Time (เวลาที่เหลือ) ของงานที่รันอยู่** — งานใหม่สั้นกว่า = ขัดทันที

1. 0–2: P1 รันไป 2 (เหลือ 5) — P2 มาถึง (BT=4) → เทียบ 4 < 5 → **ขัด P1**
2. 2–4: P2 รันไป 2 (เหลือ 2) — P3 มาถึง (BT=1) → 1 < 2 → **ขัด P2**
3. 4–5: P3 จบที่ 5 พอดี + P4 มาถึง (BT=4) → คิว: P1(5), P2(2), P4(4) → รัน P2 จบที่ 7
4. 7: เหลือ P1(5), P4(4) → P4 จบที่ 11 → P1 เศษ 5 จบที่ 16

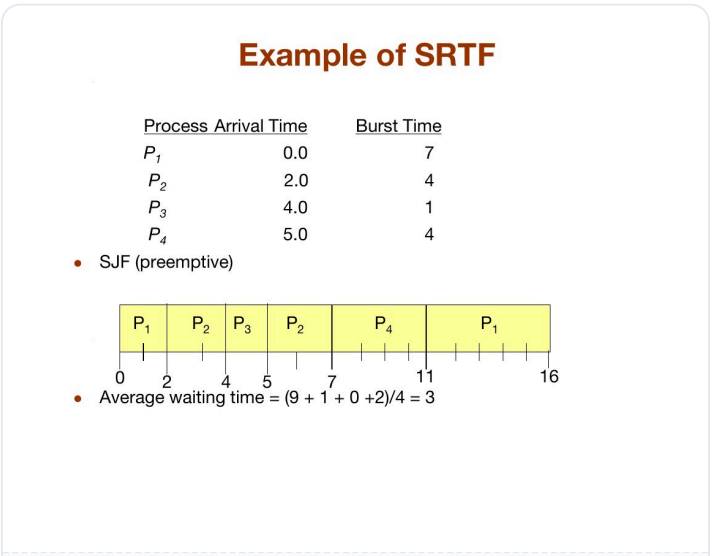


Process	Completion	TAT = C − A	WT = TAT − BT
P1	16	16	16−7 = **9**
P2	7	5	5−4 = **1**
P3	5	1	1−1 = **0**
P4	11	6	6−4 = **2**
เฉลี่ย WT	**—**		**(9+1+0+2)/4 = 3**

> 🎯 **จุดหลอก SRTF**
> เทียบด้วย **Remaining** เสมอ ไม่ใช่ BT ดั้งเดิม! (ตอนเวลา 4 ต้องเทียบ "P2 เหลือ 2" ไม่ใช่ "P2 = 4") · SRTF overhead มากกว่า SJF เพราะ context switch บ่อย



สไลด์ p.38 SRTF นิยาม



สไลด์ p.39 SRTF example (เฉลยตรงกับตารางข้างบน)

C5. Priority (ลำดับสิทธิ์) Scheduling (การจัดคิว)

- CPU ให้งาน priority สูงสุดก่อน (มีทั้ง preemptive/non-preemptive) · SJF = priority ที่ใช้ "predicted next CPU burst" เป็น priority
- **ปัญหา Starvation (การอดตาย (ไม่ได้รันเลย))** (งานสิทธิต่ำไม่ได้รัน) → **แก้ด้วย Aging**: ยิ่งรอนานยิ่งเพิ่ม priority
- **ทิศทางตัวเลข:** Unix-based → **เลขน้อย = สิทธิสูง** / Java Thread → **เลขมาก = สูง** (10 สูงสุด, default 5)

C6. Round Robin (RR) — preemptive, ตอบสนองไว

แต่ละงานรันไม่เกิน **Time Quantum (q)** หมดเวลายังไม่เสร็จ → ถูกตัดไปต่อท้ายคิว · คิวมี n งาน → ไม่มีงานรอเกิน **(n−1)q**

โจทย์ (p.41–42): ข้อมูล FCFS เดิม (P1: 0,24 · P2: 2,3 · P3: 4,3) q = 4



Process	TAT = C − A	WT = TAT − BT
P1	30 − 0 = 30	30 − 24 = **6**
P2	7 − 2 = 5	5 − 3 = **2**
P3	10 − 4 = 6	6 − 3 = **3**
เฉลี่ย WT	**—**	**11/3 ≈ 3.67**

⚖️ ขนาด quantum

q ใหญ่มาก → RR กลายเป็น **FCFS** · q เล็กมาก → **context switch ถี่เกิน overhead โต** (ช่วง switch = pure overhead)

ไทม์ไลน์ RR ทีละจังหวะ

0–4 P1(เหลือ 20) → 4–7 P2 จบ → 7–10 P3 จบ → 10–30 P1 ตัวเดียว รันรอบละ 4 จนครบ 24

Round Robin (RR)

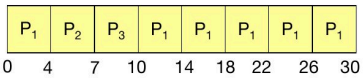
- Every process is allocated a small amount of CPU time, typically ranging from 10 to 100 milliseconds, known as a time quantum.
- Once this time has passed, the process is interrupted and placed at the end of the ready queue
- If the ready queue contains n processes and the time quantum is q, each process is allocated no more than q time units at a time. No process waits more than $(n\text{-}1)q$ time units.
- Performance
  - q large ⇒ FIFO
  - q small ⇒ q must be large with respect to context switch, otherwise overhead is too high

สไลด์ p.41 Round Robin

Example of RR with Time Quantum = 4

Process	Arrival Time	Burst Time
P_1	0.0	24
P_2	2.0	3
P_3	4.0	3

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better *response*

สไลด์ p.42 Example RR q=4 (เฉลยตรงกับตาราง)

C7. Multilevel Queue (คิวหลายชั้น) & Multilevel Feedback Queue (MLFQ)

- **Multilevel Queue**: แยก ready queue หลายคิว (เช่น foreground=RR / background=FCFS) — **งานอยู่คิวไหนตลอดชีวิต ย้ายไม่ได้** · ระหว่างคิว: fixed priority (คิวหน้าจบก่อน — เสี่ยง starvation) หรือแบ่งเวลา (80/20)
- **MLFQ**: **งานย้ายคิวได้** → ทำ aging ได้ จัดกลุ่มตามพฤติกรรมได้

โมเดลตัวอย่าง (p.46–47): Q0 = RR q8ms → ไม่จบ ลง Q1 = RR q16ms → ยังไม่จบ ลง Q2 = FCFS จนจบ · คิวล่างได้รันเมื่อคิวบนว่างหมด

Multilevel Queue Scheduling

- Multilevel Queue Scheduling is a scheduling algorithm that partitions the ready queue into several separate queues.
- Each queue can have its own scheduling algorithm. For Examples
  - foreground – RR
  - background – FCFS
- This method is used when processes can be easily categorized into different classes with different scheduling needs.
- No Movement Between Queues: Processes do not move between queues. Once assigned to a queue, a process stays there.

สไลด์ p.44 Multilevel Queue

Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – RR with time quantum 8 milliseconds
  - Q_1 – RR time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.

สไลด์ p.47 Example MLFQ 3 คิว

🎯 สรุป 5 จุดพลาดของบทนี้

1) ลบ Arrival ไม่ครบ (เอาเวลาเริ่มรันตอบเป็น WT) 2) WT ใน preemptive ไล่นับมือผิด → ใช้ WT=TAT−BT 3) ตี priority ผิดฝั่ง 4) SRTF ใช้ BT ดิบแทน remaining 5) เลือก quantum ผิด